Below is a **complete interview + real-time oriented table** that explains:

✅ **All internal data structures (data types) used inside Java Collection Framework**
✅ **How each collection class is implemented internally**
✅ **Differences between all collection classes**
✅ **When to use what (real-time usage)**

✅ 1) Internal Data Types / Data Structures used inside Java Collections (Core)

Internal Data Type / DS Used	Used In Which Java Collections	Purpose / Why used	Notes
Dynamic Array (Resizable Array)	ArrayList, Vector, Stack	Fast random access (get(i)), cache-friendly	Resizes by growing capacity
Singly Linked List	LinkedList (conceptually supports both)	Easy insert/delete at ends	Traversal is slow
Doubly Linked List	LinkedList, ArrayDeque (has array but acts like deque)	Insert/delete at both ends	LinkedList uses nodes (prev/next)
Hash Table (bucket array)	HashMap, HashSet, Hashtable, LinkedHashMap, LinkedHashSet, ConcurrentHashMap	Fast search/insert/delete using hashing	Average O(1)
Buckets + Linked List	HashMap	Collision handling	Old-style method (before Java 8)
Buckets + Balanced Tree (Red-Black Tree)	HashMap, HashSet (Java 8+)	Faster collision handling when too many collisions	Converts bucket list → tree when size exceeds threshold
Red-Black Tree	TreeMap, TreeSet	Maintains sorted order	Operations O(log n)
Heap (Binary Heap / Priority Heap)	PriorityQueue, PriorityBlockingQueue	Fast retrieval of min/max element	Insert/remove O(log n)
Circular Array / Ring Buffer	ArrayDeque	Efficient queue/deque	Faster than LinkedList

Skip List	ConcurrentSkipListMap, ConcurrentSkipListSet	Sorted + concurrent alternative to TreeMap	O(log n) but better for concurrency
Copy-On-Write Array	CopyOnWriteArrayList, CopyOnWriteArraySet	Thread-safe iteration without locking	Writes are costly
Weak Reference Table	WeakHashMap	Auto-clean keys when GC occurs	Used in caching
Identity Hash Table	IdentityHashMap	Uses == instead of .equals()	Special usage (rare)

✅ 2) Implementation of Each Java Collection Class (Internal Working)

✅ List Implementations

Collection Class	Type	Internal Implementation	Order Maintained?	Duplicate Allowed?	Null Allowed?	Time Complexity (Major)	Best Use Case
ArrayList	List	Dynamic array (Object[])	✅ Insertion order	✅ Yes	✅ Yes	get O(1), add O(1)*, remove O(n)	Best for searching + indexing
LinkedList	List, Deque	Doubly linked list (Nodes)	✅ Yes	✅ Yes	✅ Yes	add/remove ends O(1), search O(n)	Best for frequent insert/delete
Vector	List	Dynamic array + synchronized	✅ Yes	✅ Yes	✅ Yes	Like ArrayList but slower	Legacy thread-safe list
Stack	List	Vector based	✅ Yes	✅ Yes	✅ Yes	push/pop O(1)	Legacy LIFO stack

✅ Real-time rule:
- Prefer **ArrayList** over Vector
- Prefer **ArrayDeque** over Stack

✅ Set Implementations

Collection Class	Type	Internal Implementation	Order	Duplicate Allowed?	Null Allowed?	Time Complexity	Best Use Case
HashSet	Set	HashMap backing	❌ No	❌ No	✅ One null	add/remove/search O(1) avg	Best for uniqueness fast
LinkedHashSet	Set	HashMap + doubly linked list	✅ Insertion order	❌ No	✅ One null	O(1) avg	Uniqueness + preserve order
TreeSet	NavigableSet	TreeMap backing (Red Black Tree)	✅ Sorted	❌ No	❌ No (cannot sort null)	O(log n)	Need sorted unique data
EnumSet	Set	Bit vector (bitmask)	✅ Natural Enum order	❌ No	❌ No	O(1)	Fast enum storage

---

✅ Queue / Deque Implementations

Collection Class	Type	Internal Implementation	Order	Null Allowed?	Time Complexity	Best Use Case
ArrayDeque	Deque	Circular resizable array	✅ Yes	❌ No	add/remove ends O(1)	Best queue & stack replacement

PriorityQueue	Queue	Binary heap	✅ Sorted by priority	❌ No	add/remove O(log n)	Scheduling / top K elements
LinkedList	Deque	Doubly linked list	✅ Yes	✅ Yes	O(1) ends	Queue but slower than ArrayDeque

---

✅ 3) Map Implementations (Most Important)

Map Class	Internal Implementation	Order Maintained?	Allows Null Key?	Allows Null Values?	Performance	Best Use Case
HashMap	Hash table + bucket list/tree	❌ No	✅ Yes (1)	✅ Yes	O(1) avg	Most used general map
LinkedHashMap	HashMap + doubly linked list	✅ Insertion / access order	✅ Yes	✅ Yes	O(1) avg	LRU Cache, ordered map
TreeMap	Red-Black Tree	✅ Sorted order	❌ No	✅ Yes	O(log n)	Sorted key-value storage
Hashtable	Synchronized Hash table	❌ No	❌ No	❌ No	slower	Legacy thread-safe map
ConcurrentHashMap	Concurrent hash table	❌ No	❌ No	❌ No	high performance	Best thread-safe map
WeakHashMap	HashMap + WeakReference keys	❌ No	✅ Yes	✅ Yes	O(1) avg	Memory-sensitive caches
IdentityHashMap	Hash table using ==	❌ No	✅ Yes	✅ Yes	O(1) avg	Reference-key mapping

---

✅ 4) Differences Between All Major Java Collection Classes (One Master Table)

Category	Best Class	Why best	Avoid / Not recommended
Fast indexing + iteration	✅ ArrayList	O(1) get, fast iteration	LinkedList for indexing
Frequent insert/delete middle	✅ LinkedList	Node-based insertion	ArrayList shifting cost
Stack	✅ ArrayDeque	Faster + modern	Stack class
Queue	✅ ArrayDeque	Efficient FIFO	LinkedList slower
Min/Max priority scheduling	✅ PriorityQueue	Heap based priority	ArrayList manual sort
Uniqueness fast	✅ HashSet	hashing gives O(1)	TreeSet unless sorting needed
Unique + preserve insertion order	✅ LinkedHashSet	Hash + order	HashSet loses order
Unique sorted	✅ TreeSet	tree sorting	HashSet unsorted
Key-value fast	✅ HashMap	O(1)	TreeMap unless sorted required
Sorted map	✅ TreeMap	O(log n), sorted keys	HashMap unsorted
Thread-safe map	✅ ConcurrentHashMap	concurrency optimized	Hashtable
Thread-safe list	✅ CopyOnWriteArrayList	safe iteration	Vector (slow)

✅ 5) When to Use What (Real-Time Software Examples)

Requirement	Use This Collection	Example Real Project Case
Store user list and search by index	ArrayList	Employee list UI page
Insert/delete frequently	LinkedList	Chat message stream operations

Remove duplicates quickly	HashSet	Unique emails / unique userIds
Maintain insertion order with uniqueness	LinkedHashSet	Recently viewed products list
Need sorted unique elements	TreeSet	Leaderboard unique sorted scores
Key-value fast lookup	HashMap	userId → user profile
Maintain sorted keys	TreeMap	Date → events
LRU Cache	LinkedHashMap (accessOrder=true)	API response caching
Task scheduling by priority	PriorityQueue	CPU job scheduler
High concurrency updates	ConcurrentHashMap	Session storage multi-thread app
Enum keys fast	EnumMap, EnumSet	App roles and permissions

If you want, I can also provide:

✅ **Java Collection Framework hierarchy diagram** (List/Set/Queue/Map)
✅ **Interview Q&A (0–3 years)** based on these tables
✅ **Programs showing internal behavior** (HashMap collision, resize, Treeify, fail-fast iterator, etc.)